
Taller arquitectura de software
GA4-220501095-AA2-EV06



APRENDIZ
Johan Mauricio Sánchez Gaviria

INSTRUCTOR
Nelson Guzman

Análisis y desarrollo del software
FICHA
3134555

Introducción

En este documento exploraremos el concepto de arquitectura de software, su función esencial, el proceso de elaboración y cómo alcanzar una arquitectura efectiva. Finalmente, examinaremos los elementos clave del diseño de una arquitectura de software.

- ¿Qué entiende por arquitectura de software?
- ¿Cuál es su función?
- ¿Cómo se elabora la arquitectura de software?
- ¿Cómo lograr una buena arquitectura?
- ¿Cuáles son los elementos de diseño de una arquitectura de software?

- **¿Qué entiende por arquitectura de software?**

La arquitectura de software se define como un conjunto organizado de patrones y abstracciones que ofrecen un marco estructurado y comprensible para la interacción con el código fuente del software. Esta estructura no solo facilita el desarrollo y mantenimiento del software, sino que también influye en su rendimiento, escalabilidad y mantenibilidad a lo largo del tiempo.

- **¿Cuál es su función?**

La función primordial de una arquitectura de software radica en emplear los conocimientos de programación para diseñar la estructura general del software, permitiendo así la incorporación de detalles adicionales en etapas posteriores. Esta práctica capacita a los equipos de desarrollo de software para visualizar el panorama completo y comenzar a elaborar prototipos que sirvan como base para el desarrollo continuo y la iteración del producto.

- **¿Cómo se elabora la arquitectura de software?**

1. Clarifica tus Requisitos:

- ✓ Inicia con una visión general utilizando mapas mentales para resumir brevemente tus requisitos. Estos son una herramienta efectiva para organizar tus ideas.
- ✓ Crea un mapa de tus requisitos funcionales, agrupando sustantivos con verbos relevantes. Por ejemplo, vincula acciones como "ver" o "editar" con elementos como "cuenta" o "perfil".
- ✓ Considera también los requisitos no funcionales y anótalos para su posterior uso, aunque algunos, como el rendimiento, pueden ser demasiado abstractos para incluirlos en el mapa mental.

2. Comienza a Conceptualizar cada Componente:

- ✓ Imagina el escenario perfecto: ¿cómo sería tu diseño ideal? Este ejercicio te ayudará a establecer una base para tu trabajo.
- ✓ Documenta las implicaciones de tus requisitos y comienza a elaborar un borrador de trabajo con tu equipo. Observa cómo pueden contradecirse o entrar en conflicto entre sí.
- ✓ No esperes que tu diseño inicial sea definitivo. Es probable que realices cambios a lo largo del proceso de diseño.

3. Divide tu Arquitectura en "Rebanadas":

- ✓ Durante la planificación de tu diseño de arquitectura, considera cómo vas a implementarlo. Dividir tu arquitectura en "rebanadas" facilita la elaboración de un plan que beneficie a los usuarios y optimice el uso de recursos de desarrollo.

4. Prototipado:

- ✓ Mantén un registro detallado de los cambios realizados durante el prototipado para evitar errores repetidos. Documenta todas tus decisiones y ajustes.
- ✓ Establece una sola fuente de información para evitar confusiones y retrasos. Un sólido control de versiones es fundamental para mantener la coherencia en la documentación.
- ✓ Utiliza diagramas para visualizar las diferencias entre versiones de prototipos y gestionar cambios de manera efectiva.

5. Identifica y Cuantifica los Requisitos No Funcionales:

- ✓ Evalúa aspectos como el desempeño, la escalabilidad, la portabilidad, la extensibilidad y el cumplimiento normativo.
- ✓ El desempeño es crucial para evaluar el rendimiento global del sistema y de sus componentes.
- ✓ La escalabilidad se refiere a la capacidad del sistema para adaptarse a las necesidades presentes y futuras.
- ✓ La portabilidad aborda la transferibilidad de datos y componentes del sistema.
- ✓ La extensibilidad considera la capacidad de crecimiento futuro del sistema y su adaptabilidad.
- ✓ El cumplimiento normativo es esencial para garantizar que el diseño del proyecto cumpla con los estándares legales y regulatorios aplicables.

- **¿Cómo lograr una buena arquitectura?**

La arquitectura de software abarca una amplia gama de aspectos, incluyendo el rendimiento, la usabilidad, el presupuesto, la tecnología e incluso cuestiones estéticas. Para implementarla de manera efectiva, es fundamental respaldarse en una metodología de desarrollo adecuada.

- ✓ **Establecimiento de sistemas robustos y flexibles:** Es esencial diseñar sistemas que sean robustos pero que al mismo tiempo no estén atados a DreamWorks específicos. Esta libertad garantiza una estructura estable e independiente que puede adaptarse fácilmente a los cambios futuros.
- ✓ **Optimización de la construcción y el uso de la base de datos:** La base de datos debe estar alineada con las necesidades del negocio. En lugar de adaptar el negocio a la estructura de la base de datos, es fundamental diseñarla de manera que se ajuste a los requerimientos específicos de la empresa.

- ✓ **Selección de herramientas para optimizar procesos:** Es importante identificar y utilizar las herramientas adecuadas para optimizar los procesos de desarrollo y mantenimiento del software. Esto puede incluir la integración de servicios de automatización de sistemas y aplicaciones, colocándolos estratégicamente desde el inicio del proyecto para garantizar una implementación eficiente.
- ✓ **Enfoque en la auditabilidad y testabilidad:** Se debe crear un entorno donde el código pueda ser fácilmente evaluado, medido y regulado. La capacidad de auditar y probar el código de manera efectiva es fundamental para garantizar la calidad y la seguridad del software.

Al integrar estos principios en el proceso de desarrollo de software, se puede garantizar una arquitectura sólida y adaptable que cumpla con los requisitos del negocio y los usuarios finales.

- **¿Cuáles son los elementos de diseño de una arquitectura de software?**

Dentro de los componentes fundamentales de la arquitectura de software, se destacan diversas interacciones que son esenciales para su funcionamiento:

- ✓ **Llamadas a procedimientos:** Estas interacciones representan la ejecución de funciones o métodos dentro del software para realizar acciones específicas. Las llamadas a procedimientos son la base de la lógica de programación y permiten la ejecución de tareas definidas en el código.
- ✓ **Comportamiento de variables:** Las variables juegan un papel crucial en la arquitectura de software al almacenar y manipular datos. Su comportamiento determina cómo se almacenan, modifican y utilizan los valores a lo largo del programa. Esto incluye aspectos como el alcance, el tipo de datos y la gestión de la memoria.
- ✓ **Protocolos cliente-servidor:** Estos protocolos establecen la comunicación entre clientes y servidores en un entorno de red. Facilitan el intercambio de información y la coordinación de acciones entre diferentes dispositivos o sistemas distribuidos. Los protocolos cliente-servidor son fundamentales para aplicaciones que requieren acceso remoto a recursos o servicios.
- ✓ **Transmisión asíncrona de eventos:** La transmisión asíncrona de eventos permite que los componentes del software se comuniquen de manera independiente y asíncrona. Esto significa que los eventos pueden ser enviados y recibidos en momentos diferentes, lo que proporciona flexibilidad y eficiencia en la gestión de eventos en tiempo real.

Estos componentes interactúan entre sí para formar la estructura y el funcionamiento del software, creando una base sólida para el desarrollo de aplicaciones eficientes y robustas.